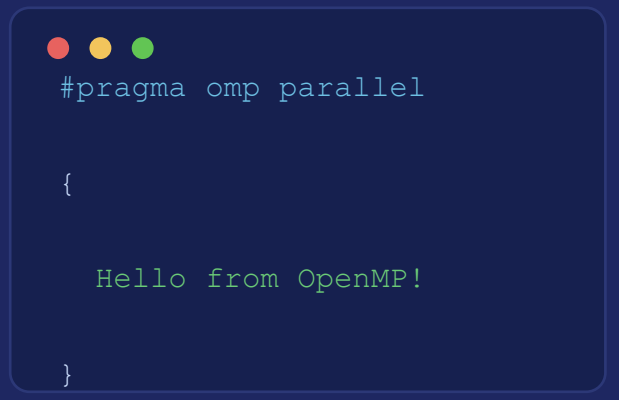
CS-401 · PARALLEL AND DISTRIBUTED COMPUTING · LECTURE 4 COMPANION

OpenMP Programming Lab

A hands-on Google Colab walkthrough: from your first parallel program to a working reduction



Dr. Tariq Sadad

Department of Computer Science · IIUI

LAB ROADMAP

What you'll build today

1

Set up Colab

Open a fresh Google Colab notebook — no local install needed.

2

Run your first program

Write, compile and run a parallel "Hello, OpenMP" in one cell.

3

Meet the threads

Print each thread's ID and control how many threads run.

4

Parallel sum

Split an array across threads and safely combine the results.

STEP 1

Open Google Colab

1

Go to colab.research.google.com

Sign in with any Google account — nothing to install.

2

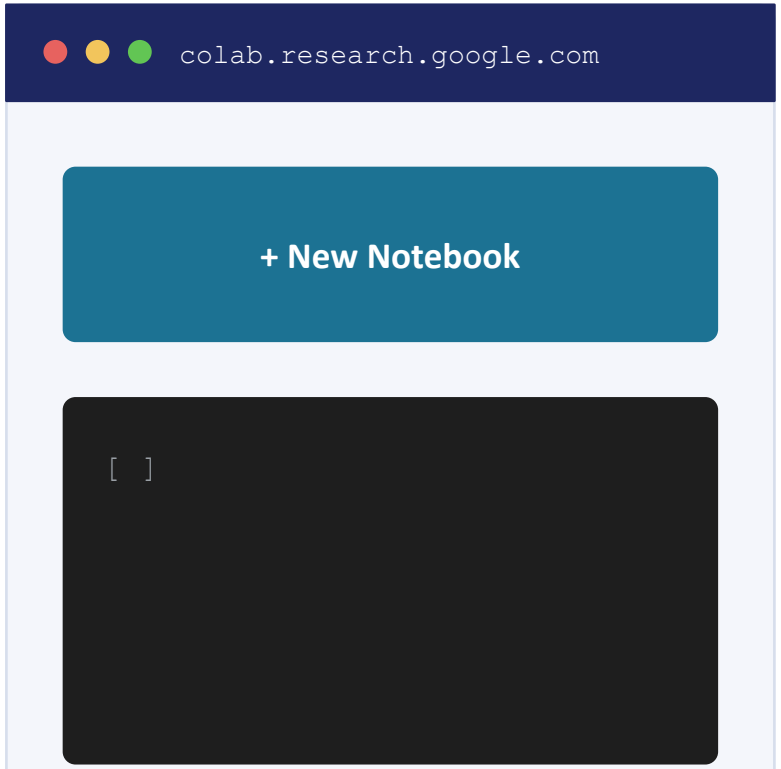
Click “New Notebook”

This opens a blank notebook with one empty code cell.

3

You're ready to code

Every OpenMP program in this lab is written and run inside these cells.



Empty code cell — ready for input

STEP 2 · FIRST PROGRAM

Hello from OpenMP

Copy this into the first cell and run it (▶).

hello.cpp

```
%%writefile hello.cpp
#include <iostream>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        std::cout << "Hello from OpenMP!" << std::endl;
    }
    return 0;
}
```

STEP 2 · COMPILE & RUN

Build it, run it, see it

terminal

```
!g++ -fopenmp hello.cpp -o hello
!./hello

Hello from OpenMP!
Hello from OpenMP!
Hello from OpenMP!
Hello from OpenMP!
```

Why does it print four times?

`#pragma omp parallel` creates a **team of threads**.

Every thread in the team runs the same block of code — the one inside the braces — independently and (roughly) at the same time.

On this machine, OpenMP started 4 threads, so the line was printed 4 times.

STEP 3 · THREAD IDENTITY

Give each thread a name

Replace the first program with this one, in a new cell.

threads.cpp

```
%%writefile threads.cpp
#include <iostream>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        std::cout << "Thread number: " << id << std::endl;
    }
    return 0;
}
```

STEP 3 · RUN WITH 4 THREADS

Request threads explicitly

terminal

```
!g++ -fopenmp threads.cpp -o threads
!OMP_NUM_THREADS=4 ./threads

Thread number: 0
Thread number: 2
Thread number: 1
Thread number: 3
```

Order isn't guaranteed — threads run independently, so any order (or interleaving) is valid.

Try it live

Change the thread count and re-run:

`OMP_NUM_THREADS=2`	→ *2 lines printed*
`OMP_NUM_THREADS=8`	→ *8 lines printed*
`OMP_NUM_THREADS=1`	→ *back to serial*

CHECKPOINT

Remember these three things

```
#pragma omp parallel
```

Creates a team of threads that all execute the enclosed block.

```
omp_get_thread_num()
```

Tells each thread its own ID, starting at 0.

```
OMP_NUM_THREADS=4
```

An environment variable that requests four threads at run time.

STEP 4 · PRACTICAL EXERCISE

Add numbers in parallel

Four numbers, one sum — split the work across threads instead of adding one at a time.



`reduction(+:sum)`
combines partial sums safely

sum = 100

STEP 4 · CODE

sum.cpp

sum.cpp

```
%%writefile sum.cpp
#include <iostream>
#include <omp.h>

int main() {
    int sum = 0;
    int a[4] = {10, 20, 30, 40};

    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < 4; i++) {
        sum += a[i];
    }

    std::cout << "Total sum = " << sum << std::endl;
    return 0;
}
```

STEP 4 · RUN & UNDERSTAND

Two directives doing the work

terminal

```
!g++ -fopenmp sum.cpp -o sum
!OMP_NUM_THREADS=4 ./sum

Total sum = 100
```

parallel for

distributes loop iterations across the thread team.

reduction(+:sum)

gives each thread its own private copy of **sum**, then safely adds all the copies together at the end — no race condition, no manual locking.

Without reduction(), multiple threads writing to the same sum at once would corrupt the result.

INSTRUCTOR NOTES

Running this as an in-class activity

1

Demonstrate

Live-run Step 2 on the projector so students see the output before touching code.

2

Experiment

Ask students to change OMP_NUM_THREADS in Step 3 and predict the output first.

3

Practice

Assign Step 4 as a hands-on exercise — have them modify the array or add a 5th thread.

KEY TAKEAWAYS

From pragma to parallel sum

1. A single #pragma omp parallel turns a block of code into work every thread runs.
2. omp_get_thread_num() and OMP_NUM_THREADS give you visibility and control over the thread team.
3. parallel for hands out loop iterations to threads; reduction() combines their results safely.
4. Next: try more threads, bigger arrays, or a different operator inside reduction().